

Android 中使用 CircleImageView 实现圆形图片

Android XML 中使用第三方库详解（以 circleimageview 为例）

在 Android 开发中，第三方库能极大简化开发流程（如 `circleimageview` 可快速实现圆形图片，无需手动写自定义 View）。以下是 **从依赖引入到 XML 布局使用、Java 逻辑控制、高级配置** 的完整步骤，结合实际场景详细讲解。

一、核心概念：第三方库的引入方式

Android 项目引入第三方库主要通过 **Gradle 依赖**（最常用），原理是通过 Maven 仓库（如 Google、JCenter、Maven Central）下载库的编译产物（.aar 或 .jar），自动集成到项目中。

`de.hdodenhof:circleimageview` 是一个轻量级库，专门用于显示圆形图片，支持边框、圆角（可配置为椭圆）等功能，GitHub 地址：[circleimageview](#)

二、步骤 1：环境准备（确认仓库配置）

首先确保项目的 Gradle 配置能访问到第三方库所在的 Maven 仓库（`circleimageview` 托管在 Maven Central）。

1. 检查项目级 build.gradle（Project 根目录）

确保 `repositories` 中包含 `mavenCentral()`（Android Studio 4.2+ 默认已配置，低版本需手动添加）：

Code block

```
1 // 项目级 build.gradle (build.gradle (Project: 你的项目名))
2 buildscript {
3     repositories {
4         google() // 谷歌官方仓库
5         mavenCentral() // 关键: Maven Central 仓库 (circleimageview 在这里)
6     }
7     dependencies {
8         classpath "com.android.tools.build:gradle:7.0.0" // 你的 Gradle 插件版本
9     }
10 }
11
12 allprojects {
13     repositories {
```

```
14         google()
15         mavenCentral() // 同样需要添加这里
16     }
17 }
```

2. 检查 Gradle 版本兼容性

`circleimageview` 3.0+ 版本要求 Android Gradle Plugin (AGP) 4.0+，如果你的 AGP 版本较低（如 3.x），需使用 2.x 版本的 `circleimageview`（后续会说明）。

三、步骤 2：添加依赖（Module 级 build.gradle）

核心步骤：在 **模块级 build.gradle**（通常是 `app` 模块下的 `build.gradle (Module: app)`）中添加依赖，同步后自动下载库。

1. 最新版本依赖（推荐）

打开 [circleimageview 的 GitHub Releases](#)，查看最新版本（当前最新是 3.1.0），添加依赖：

Code block

```
1 // 模块级 build.gradle (build.gradle (Module: app))
2 dependencies {
3     // 其他依赖 (如 Material Design、AppCompat 等)
4     implementation 'androidx.appcompat:appcompat:1.6.1'
5     implementation 'com.google.android.material:material:1.12.0'
6
7     // 关键：添加 circleimageview 依赖
8     implementation 'de.hdodenhof:circleimageview:3.1.0'
9 }
```

2. 低版本兼容（AGP < 4.0）

如果你的项目 AGP 版本低于 4.0（如 3.6.x），3.x 版本的 `circleimageview` 会报错，需使用 2.2.0 版本（兼容旧版）：

Code block

```
1 implementation 'de.hdodenhof:circleimageview:2.2.0'
```

3. 同步依赖（Sync Now）

添加依赖后，点击 Android Studio 右上角的 **Sync Now** 按钮，Gradle 会自动从 Maven Central 下载 `circleimageview` 的 .aar 文件，并集成到项目中。

- 同步成功后，可在 `External Libraries` 中看到 `de.hdodenhof:circleimageview:3.1.0`，说明库已引入。
- 若同步失败：检查网络（需能访问 Maven Central）、仓库配置是否正确、依赖版本号是否存在。

四、步骤 3：XML 布局中使用 CircleImageView

依赖同步成功后，即可像使用原生 `ImageView` 一样，在 XML 布局中直接使用 `CircleImageView`，并通过自定义属性配置样式。

1. 基础用法（圆形图片）

最简化配置：仅设置宽高、图片资源（src），默认显示圆形图片：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:gravity="center"
7      android:background="@color/white">
8
9      <!-- 基础圆形图片：宽高必须一致（否则会变成椭圆） -->
10     <de.hdodenhof.circleimageview.CircleImageView
11         android:id="@+id/civ_avatar"
12         android:layout_width="120dp" <!-- 宽高需相等，保证圆形 -->
13         android:layout_height="120dp"
14         android:src="@mipmap/ic_avatar" <!-- 图片资源 (mipmap/drawable 均可) --
15     >
16         android:scaleType="centerCrop" <!-- 图片缩放模式（同 ImageView） -->
17         android:contentDescription="@string/avatar" /> <!-- 无障碍描述（必填） --
18     </de.hdodenhof.circleimageview.CircleImageView>
19 </LinearLayout>
```

关键注意点：

- **宽高必须相等**：如果 `layout_width` 和 `layout_height` 不一致，图片会变成椭圆（如需椭圆可刻意设置不等）。
- **scaleType 推荐**：使用 `centerCrop`（裁剪图片居中显示，填满控件），避免图片拉伸变形。
- **contentDescription**：无障碍必备属性，需在 `strings.xml` 中定义：

Code block

```
1 <!-- res/values/strings.xml -->
2 <string name="avatar">用户头像</string>
```

2. 高级配置（边框、阴影、圆角等）

`circleimageview` 提供了专属自定义属性（需在 XML 根布局添加命名空间），支持边框颜色、边框宽度、阴影、圆角（3.0+ 版本）等。

步骤 1：添加自定义属性命名空间

在布局文件的根标签中，添加 `xmlns:app="http://schemas.android.com/apk/res-auto"`（用于引用库的自定义属性）：

Code block

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto" <!-- 关键：添加命名空间 -->
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     android:gravity="center"
8     android:background="@color/white">
9
10    <!-- 带边框、阴影的圆形图片 -->
11    <de.hdodenhof.circleimageview.CircleImageView
12        android:id="@+id/civ_avatar_advanced"
13        android:layout_width="120dp"
14        android:layout_height="120dp"
15        android:src="@mipmap/ic_avatar"
16        android:scaleType="centerCrop"
17        android:contentDescription="@string/avatar"
18
19        <!-- 1. 边框配置 -->
20        app:civ_border_color="@color/primary" <!-- 边框颜色（需在 colors.xml 定义） -->
21        app:civ_border_width="2dp" <!-- 边框宽度（dp 单位） -->
22        app:civ_border_overlay="false" <!-- 边框是否覆盖图片（默认 false：边框在图片外侧） -->
23
24        <!-- 2. 阴影配置（仅 API 21+ 有效） -->
25        app:civ_shadow_color="@color/gray_300" <!-- 阴影颜色 -->
26        app:civ_shadow_radius="4dp" <!-- 阴影模糊半径 -->
27        app:civ_shadow_offset="0dp" <!-- 阴影偏移量（x/y 方向，默认 0） -->
```

```
28         app:civ_shadow_ambient_color="@color/gray_200"    <!-- 环境阴影颜色 (3.0+
新增) -->
29
30         <!-- 3. 圆角配置 (3.0+ 新增, 实现椭圆或圆角矩形) -->
31         app:civ_corner_radius="16dp"                      <!-- 圆角半径 (仅当宽高相等时, 圆角
+圆形=圆角圆形) -->
32         app:civ_oval="true" />                            <!-- 是否强制椭圆 (默认 true: 宽高
相等时为圆形) -->
33
34     </LinearLayout>
```

自定义属性详解（核心）

属性名	作用	类型	默认值
civ_border_color	边框颜色	color	透明
civ_border_width	边框宽度	dimension	0dp
civ_border_overlay	边框是否覆盖图片（false=外侧，true=覆盖）	boolean	false
civ_shadow_color	阴影颜色（API 21+）	color	透明
civ_shadow_radius	阴影模糊半径（API 21+）	dimension	0dp
civ_corner_radius	圆角半径（3.0+）	dimension	0dp
civ_oval	是否强制椭圆（false=圆角矩形）	boolean	true

示例效果说明：

- 若 `civ_oval="true"` + 宽高相等 + `civ_corner_radius="16dp"` → 圆角圆形图片（类似微信头像）。
- 若 `civ_oval="false"` + 宽高不等 + `civ_corner_radius="16dp"` → 圆角矩形图片。
- 若 `civ_border_overlay="true"` → 边框会覆盖在图片边缘（适合细边框）。

五、步骤 4：Java/Kotlin 代码中控制 CircleImageView

除了 XML 配置，还可在代码中动态修改 `CircleImageView` 的属性（如图片资源、边框颜色、阴影等），用法与原生 `ImageView` 类似，但需调用 `circleimageview` 提供的专属方法。

1. 初始化与设置图片资源

```

1  import android.os.Bundle;
2  import androidx.appcompat.app.AppCompatActivity;
3  import de.hdodenhof.circleimageview.CircleImageView;
4
5  public class AvatarActivity extends AppCompatActivity {
6
7      private CircleImageView civAvatar;
8
9      @Override
10     protected void onCreate(Bundle savedInstanceState) {
11         super.onCreate(savedInstanceState);
12         setContentView(R.layout.activity_avatar);
13
14         // 1. 绑定 XML 中的 CircleImageView
15         civAvatar = findViewById(R.id.civ_avatar_advanced);
16
17         // 2. 动态设置图片资源 (3种方式)
18         // 方式1: 设置 mipmap/drawable 中的资源
19         civAvatar.setImageResource(R.mipmap.ic_avatar_new);
20
21         // 方式2: 设置网络图片 (需配合 Glide/Picasso 等图片加载库)
22         // Glide.with(this).load("https://xxx.com/avatar.jpg").into(civAvatar);
23
24         // 方式3: 设置 Bitmap (如拍照/裁剪后的图片)
25         // Bitmap bitmap = BitmapFactory.decodeFile("/sdcard/avatar.jpg");
26         // civAvatar.setImageBitmap(bitmap);
27     }
28 }

```

2. 动态修改自定义属性（边框、阴影等）

`CircleImageView` 提供了对应的 `setter` 方法，可在代码中动态修改：

Code block

```

1  // 1. 修改边框颜色和宽度
2  civAvatar.setBorderColor(getResources().getColor(R.color.red)); // 边框颜色
3  civAvatar.setBorderWidth(3); // 边框宽度 (单位: px, 需注意 dp 转 px)
4
5  // 2. 修改阴影 (API 21+)
6  civAvatar.setShadowColor(getResources().getColor(R.color.black));
7  civAvatar.setShadowRadius(6); // 阴影模糊半径 (px)
8
9  // 3. 修改圆角半径 (3.0+)
10 civAvatar.setCornerRadius(24); // 圆角半径 (px)
11
12 // 4. 切换椭圆/圆角矩形模式

```

```
13  civAvatar.setOval(false); // 改为圆角矩形
14
15  // 5. 设置边框是否覆盖图片
16  civAvatar.setBorderOverlay(true);
```

注意：dp 转 px 工具（避免适配问题）

代码中设置 `borderWidth`、`shadowRadius` 等属性时，方法参数是 **px 单位**，而 XML 中用的是 dp，需手动转换 dp 为 px：

Code block

```
1  /**
2   * dp 转 px 工具方法
3   */
4  public int dp2px(float dpValue) {
5      final float scale = getResources().getDisplayMetrics().density;
6      return (int) (dpValue * scale + 0.5f); // 四舍五入
7  }
8
9  // 使用示例：设置 2dp 边框
10  civAvatar.setBorderWidth(dp2px(2));
```

3. 配合图片加载库（Glide/Picasso）

实际开发中，网络图片加载常用 `Glide` 或 `Picasso`，直接传入 `CircleImageView` 即可（无需额外配置，库会自动适配圆形裁剪）：

Code block

```
1  // 1. 添加 Glide 依赖（模块级 build.gradle）
2  // implementation 'com.github.bumptech.glide:glide:4.16.0'
3  // annotationProcessor 'com.github.bumptech.glide:compiler:4.16.0'
4
5  // 2. 用 Glide 加载网络图片到 CircleImageView
6  Glide.with(this)
7      .load("https://example.com/user_avatar.jpg") // 网络图片地址
8      .placeholder(R.mipmap.ic_avatar_default) // 加载中占位图
9      .error(R.mipmap.ic_avatar_error) // 加载失败占位图
10     .centerCrop() // 裁剪模式
11     .into(civAvatar); // 目标 CircleImageView
```

六、常见问题与解决方案

1. 依赖同步失败

- 问题：Sync 时提示 `Failed to resolve: de.hdodenhof:circleimageview:3.1.0`
- 解决方案：
 - a. 检查网络是否能访问 Maven Central（可手动打开 <https://search.maven.org/> 验证）。
 - b. 确认项目级和模块级 `build.gradle` 中都添加了 `mavenCentral()`。
 - c. 清除 Gradle 缓存：File → Invalidate Caches... → 勾选 "Clear file system cache and local history" → 点击 Invalidate and Restart。

2. 图片显示为椭圆而非圆形

- 问题：`CircleImageView` 显示椭圆，不是圆形。
- 解决方案：确保 `layout_width` 和 `layout_height` 数值相等（如均为 120dp）。

3. 边框不显示

- 问题：设置了 `civ_border_color` 和 `civ_border_width`，但边框不显示。
- 解决方案：
 - a. 检查边框颜色是否为透明（如 `@color/transparent`）。
 - b. 确保 `civ_border_width` 数值大于 0（如 2dp，不要设为 0）。
 - c. 若 `civ_border_overlay="true"`，边框可能被图片覆盖，改为 `false` 试试。

4. 阴影不显示

- 问题：设置了阴影属性，但看不到阴影。
- 解决方案：
 - a. 阴影仅支持 API 21+（Android 5.0+），低版本设备不显示。
 - b. 确保 `civ_shadow_radius` 大于 0（如 4dp），且阴影颜色不是透明。
 - c. 控件需留出阴影空间（如设置 `layout_margin`，避免阴影被父布局裁剪）。

七、扩展：其他类似第三方库推荐

如果 `circleimageview` 满足不了需求，可尝试以下替代库：

1. **Glide Transformations**：配合 Glide 使用，支持圆形、圆角、高斯模糊等多种图片变换（无需额外 ImageView）。
2. **ShapeableImageView**（Material Design 组件）：Google 官方提供的可自定义形状的 ImageView，支持圆形、圆角、三角形等，无需第三方库（需引入 Material Design 依赖）。

总结

使用第三方库的核心流程是：**配置仓库** → **添加依赖** → **XML 布局使用** → **代码控制**。以 `circleimageview` 为例，关键点：

1. 依赖引入需确保仓库和版本兼容。
2. XML 中需添加 `xmlns:app` 命名空间，才能使用自定义属性。
3. 宽高相等是显示圆形的前提，`scaleType="centerCrop"` 避免图片变形。
4. 代码中动态修改属性时，注意 dp 转 px 适配不同屏幕。

通过以上步骤，即可快速在项目中集成圆形图片功能，无需手动编写自定义 View，大幅提升开发效率。

（注：文档部分内容可能由 AI 生成）